CLAUDE CODE MASTERCLASS · MODULE 01

Claude Code Fundamentals

The ground floor of agentic engineering with Claude Code: the product family, installation and editor choice, the CLAUDE.md constitution, prompt caching, model selection, the permission model, git worktrees, the agents view, and artifacts.

English edition
July 2026

Contents

Claude Code Fundamentals . . . 3
Get Started with Claude Code . . . 5
Install and Configure Claude Code . . . 7
Choose Your Editor . . . 9
Write a CLAUDE.md Constitution . . . 10
Keep the Prompt Cache Warm . . . 13
Select the Right Model . . . 16
Configure the Permission Model . . . 18
Run Parallel Agents with Git Worktrees . . . 22
Supervise Background Sessions with the Agents View . . . 24
Turn Session Output into a Shareable Page . . . 27
Labs: Install, Build, and Prompt . . . 31

MODULE 01 · OVERVIEW

Claude Code Fundamentals

Claude Code is Anthropic's agentic coding tool: a command-line interface that puts a capable model directly inside your terminal, your IDE, and your CI pipeline. This module covers the ground floor. It starts with the product family and where the CLI sits in it, walks through installation and the three environments it runs in, then turns to the three things that decide the quality and the cost of every session: the context you give it (CLAUDE.md), the prefix it re-sends every turn (prompt caching), and the model you point at the task.

The second half is about control and scale. The permission model decides what Claude may touch, git worktrees keep parallel work from colliding, the agents view turns one session into a supervised fleet, and artifacts publish the result as a live page. Every topic in this module is a lever you will keep using in Modules 2 to 13.

Topics in this module

Topic	Description
01: Introduction	The Claude product family (Claude.ai, Desktop, Code CLI, IDE extensions, API, and Managed Agents) and why the terminal-first tool is the focus of this course
02: Setup	Installing Node.js and the `@anthropic-ai/claude-code` CLI, first-time authentication, and configuring the VS Code and Claude Terminal extensions
03: Editors	Running Claude Code in the terminal, VS Code sidebar, and Claude Desktop, with guidance on which environment fits which working style
04: Constitution	Context engineering with `CLAUDE.md`, the multi-layer file hierarchy, `@`-file mentions, and the built-in memory system
05: Prompt Caching	The three-layer prefix, the actions that invalidate it and the ones that safely append, TTL and scope, and reading hit rate from `/usage`
06: Model Selection	Choosing between Opus, Sonnet, and Haiku based on task complexity, and managing token usage and session cost
07: Permissions	The three permission tiers, the allow/ask/deny rule syntax, permission modes, and the settings scopes that decide which rule wins
08: Git Worktrees	How worktrees enable parallel branch checkouts, the `--worktree` flag and `EnterWorktree` tool, and how to clean them up
09: Agents View	Dispatching background sessions with `--bg`, triaging the board by state, and the supervisor that keeps sessions alive
10: Artifacts	Publishing session output as a live page on claude.ai: the permission prompt, versions and sharing, design tokens, and page constraints

Demos in this module

Demo	Description
Build a three-layer constitution	Builds a global, project, and subfolder `CLAUDE.md` from a real project map, then replaces pasted code with `@`-mentions and separates durable rules from `/memory` notes

Demo	Description
Measure the cache hit rate	Warms a cache across three turns, breaks it on purpose with a `/model` switch, proves a mid-session `CLAUDE.md` edit is inert until `/clear`, and compares `/compact` against `/rewind`
Route tasks to the right model	Runs the same tasks on Haiku, Sonnet, and Opus, measures each with `/usage` and `/context`, and writes the measured routing policy into `CLAUDE.md`
Lock down a project with settings	Builds `.claude/settings.json` with allow and deny rules, makes a secrets file unreachable by file and by shell, and splits team policy from a personal override
Run two sessions without collisions	Runs two Claude sessions in parallel worktrees editing the same file, then reviews, merges, and cleans up the branches they produce
Triage a fleet of background sessions	Dispatches three background sessions on independent chores, triages them from the board without attaching, and merges only the work that earns it
Publish and share a decision	Publishes a CI failure dashboard from a CSV, revises it through versions, restyles it from design tokens, and proves the fallback when `Artifact` is denied

Labs in this module

Lab	Track	Time	Focus
Install and Set Up Claude Code	Required	30m	Install the CLI, authenticate, and explore it through the VS Code extension
Build a Task Tracking REST API	Core	40m	Build a Python REST API end to end, prompt-driven: you write the prompt, Claude does the work
Apply Prompting Techniques and Install a Plugin	Optional	50m	Statefile maintenance, anchors, `@`-mentions, few-shot examples, slash-command promotion, then a real marketplace plugin

Further resources

- Claude Code documentation (https://code.claude.com/docs/en/overview): what the CLI is, how it is installed, and where it runs
- Anthropic model overview (https://platform.claude.com/docs/en/about-claude/models/overview): Opus, Sonnet, and Haiku compared across workloads
- Memory and CLAUDE.md (https://code.claude.com/docs/en/memory): the file hierarchy Claude reads before every session
- Prompt caching in Claude Code (https://code.claude.com/docs/en/prompt-caching): the layer model, the invalidation list, TTL and scope
- Configure permissions (https://code.claude.com/docs/en/permissions): tiers, rule syntax, modes, and settings precedence
- Worktrees (https://code.claude.com/docs/en/worktrees): isolated checkouts for parallel agents
- Agent view (https://code.claude.com/docs/en/agent-view): dispatching and supervising background sessions
- Share session output as artifacts (https://code.claude.com/docs/en/artifacts): publishing, versioning, and sharing a page

CHAPTER 1

Get Started with Claude Code

Environment: terminal, VS Code, browser · Where to find it: the `claude` command in any project directory, claude.ai in a browser, and the Claude Desktop app

Unlike a chat assistant that answers questions, Claude Code acts. It reads files, writes code, runs tests, and coordinates multi-step tasks autonomously. This course teaches you how to work with it effectively, from first run to production-grade agentic workflows.

The Claude Family

Anthropic ships Claude through a family of products, each designed for a different context and level of autonomy. They all share the same underlying models (Opus, Sonnet, Haiku) and all connect to the same Anthropic API, but their interfaces and capabilities are tuned for different use cases.

Claude.ai

Claude.ai is the web-based chat interface. It is the fastest way to have a conversation with Claude without installing anything: you log in with an email address and start chatting immediately. It covers everyday tasks such as drafting, summarizing, explaining concepts, quick code questions, and image understanding.

Claude Desktop

Claude Desktop is the native application for macOS and Windows. It offers the same conversational experience as the web app but adds desktop-class integrations: local file access, MCP server connections, and the ability to control other apps on your machine. Claude Desktop is where many developers first encounter Claude's tool-use capabilities outside the browser.

Claude Code CLI

Claude Code is the terminal-first tool and the central focus of this course. It is installed globally via npm and launched from any project directory with the `claude` command. Once running, it reads your codebase, understands the full project context, and can autonomously complete multi-file tasks (writing features, fixing bugs, running commands, and committing changes) without switching to a separate chat window.

```bash
# Start an interactive session in the current project
claude

# Run a one-shot task from the terminal
claude "Add input validation to the login form and write tests for it"
```

Claude Code for VS Code and JetBrains

The IDE extensions bring the same agentic capabilities into the editor. The Claude Code extension for VS Code integrates the chat panel, inline diff reviews, and selection-aware context directly into the sidebar. JetBrains IDEs are supported through a separate plugin. Both extensions share session state with the CLI, so you can switch between terminal and IDE without losing context.

claude.ai/code

claude.ai/code is the browser-based entry point for users who prefer not to install the CLI. It provides a hosted coding environment where Claude can work on repositories and files directly in the browser. It targets developers who want agentic coding help without a local setup, and shares the same model and tool capabilities as the CLI.

Claude for Work

Claude for Work covers the Teams and Enterprise subscription tiers. It adds collaboration features (shared projects, knowledge bases, and organization-level admin controls) on top of the standard chat experience. Enterprise deployments can connect to internal data sources and enforce the data-handling policies required by regulated industries.

Managed Agents and the Agent SDK

Managed Agents is Anthropic's platform for building and deploying production AI agents that run in the cloud. The Claude Agent SDK, available for Python and TypeScript, lets developers define custom agents, wire them to tools and data sources, and orchestrate multi-agent pipelines. This is the layer where Claude transitions from a developer tool into a software component inside a larger system.

The Anthropic API

Everything in the Claude family is powered by the Anthropic API. Direct API access is for developers who want full control: choosing models, crafting system prompts, implementing tool use, and integrating Claude into their own products. The API supports streaming, prompt caching, batch processing, and the full range of Claude's multimodal capabilities.

Why Claude Code for Developers

Claude Code occupies a unique position in the family because it operates where code lives: in the terminal, in the IDE, and in CI/CD pipelines. It has persistent access to the full repository, can invoke shell commands, and understands git history. That combination enables workflows a chat interface cannot match, including autonomous multi-file refactors, test-driven bug fixes, and end-to-end feature implementation driven by a single natural-language prompt.

Surface	For whom	Primary use
Claude.ai	Anyone	Conversation, drafting, quick questions
Claude Desktop	Knowledge workers	Chat plus local files and MCP connections
Claude Code CLI	Developers	Agentic work in a repository
IDE extensions	Developers in VS Code or JetBrains	The agent loop inside the editor
Managed Agents and Agent SDK	Platform teams	Production agents running in the cloud
Anthropic API	Product engineers	Claude as a component in your own software

CHAPTER 2

Install and Configure Claude Code

Environment: terminal, VS Code · Where to find it: any shell for the npm install, the VS Code Extensions panel for the two extensions

This chapter covers installing Claude Code and its prerequisites on macOS and Windows, configuring the VS Code integration, and verifying the environment is ready for the rest of the course.

Requirements

Claude Code runs as a Node.js CLI tool, so Node.js 18 or later is the only hard dependency. On macOS, the recommended path is Homebrew followed by `nvm` or the official Node.js installer. On Windows, download the LTS installer from nodejs.org or use `winget install OpenJS.NodeJS.LTS`.

Verify your Node.js installation before proceeding:

```bash
node --version   # must be 18 or later
npm --version
```

Installing Claude Code

Claude Code is published to npm as `@anthropic-ai/claude-code`. Install it globally so the `claude` binary is available system-wide. On Windows, open PowerShell or Command Prompt as a regular user rather than as Administrator.

```bash
npm install -g @anthropic-ai/claude-code
```

First-time Authentication

Run `claude` in any terminal to launch the interactive setup wizard. You will be prompted to authenticate either via a browser-based login or by pasting an API key generated in the Anthropic Console.

```bash
claude
```

After authentication, Claude Code stores a session token locally and will not prompt again. Run `claude --version` to confirm the installation is working correctly.

VS Code Extensions

Two extensions are recommended for working with Claude Code inside VS Code.

The Claude Code extension (publisher `anthropic`) is the official Anthropic integration. It embeds Claude Code's chat and agent interface directly in the VS Code sidebar, adds editor-aware context such as the active file and selection, and surfaces inline diff views when Claude edits code.

The Claude Terminal extension (publisher `cjnoname`) adds a dedicated terminal profile that launches the interactive CLI pre-configured for the current workspace. It removes the need to open a separate terminal window and ensures the working directory is always set to the repository root.

```
ext install anthropic.claude-code
ext install cjnoname.claude-terminal
```

Verifying the Environment

With Node.js, Claude Code, and the VS Code extensions installed, open a terminal in the root of any project and run:

```bash
claude --version
claude "What files are in this directory?"
```

A successful response confirms authentication and tool access are working.

Command	Usage
`/status`	Verify authentication is working and display the current API key and model
`/help`	Show available slash commands and their descriptions
`/doctor`	Health check the installation, including settings files that fail to parse

CHAPTER 3

Choose Your Editor

Environment: terminal, VS Code, Claude Desktop · Where to find it: a shell prompt, the VS Code sidebar, or the desktop app window

Claude Code runs in three distinct environments, each suited to a different working style. The right choice depends on how much context you want from your editor, how often you switch between coding and conversation, and whether you need a local setup at all.

Terminal

The terminal is the native home of Claude Code. After a global install, the `claude` command works in any project directory with no additional configuration: open a shell, navigate to a repository, and start working.

The terminal interface is minimal by design. It gives you the full agent loop without any IDE overhead, which makes it a good fit for remote development, SSH sessions, and scripted one-shot tasks. It is also the environment where the CLI flags and pipe-based workflows are most accessible.

VS Code

VS Code is the recommended starting environment for this course. It is free, cross-platform, and the Claude Code extension integrates the full agent interface directly into the sidebar. Because the IDE remains open alongside the agent, you see git diffs, file explorer changes, test output, and inline code decorations in real time, context that a standalone terminal window does not surface.

The extension also adds editor-aware features: Claude reads the active file and current selection automatically, and edits are shown as inline diffs before they are applied. For developers new to agentic workflows, this makes the feedback loop tangible and easy to follow.

VS Code's integrated terminal runs Claude Code just as a standalone terminal would, so you can switch between the sidebar chat and the CLI without leaving the editor. Installing both extensions gives you the most complete setup.

Claude Desktop App

Claude Desktop is the native macOS and Windows application. Unlike the terminal or the VS Code extension, it is not a coding environment. It is a conversation interface with added desktop integration: local file access, MCP server connections, and the ability to interact with other apps on your machine.

Claude Desktop is useful when you want to talk through architecture decisions, review documents, or use MCP-connected tools outside of a coding session. It shares the same underlying models as the CLI and can connect to the same MCP servers configured in your project.

Command	Usage
`/model`	Switch between Claude models mid-session
`/status`	Display the current editor environment and session info
`/help`	List all available slash commands in the current environment

CHAPTER 4

Write a CLAUDE.md Constitution

Environment: terminal, VS Code · Where to find it: `CLAUDE.md` at the repository root, `~/.claude/CLAUDE.md` for global rules, and `/memory` for evolving notes

Context engineering is the practice of deciding what Claude sees, what it remembers, and what it can reference, before and during a session. The quality of Claude's output is directly tied to the quality of its context: the right files, the right instructions, and the right level of detail at the right time.

CLAUDE.md, the Project Constitution

CLAUDE.md is a markdown file that Claude reads automatically at the start of every session. It is the primary mechanism for encoding persistent, project-level instructions: coding conventions, architectural decisions, test strategies, deployment notes, and anything else Claude should know without being told each time.

Think of it as a standing brief for a new engineer joining the project. The file is checked into the repository, so the same instructions apply to every developer and every CI run that uses Claude Code.

```markdown
# CLAUDE.md

## Hard rules

- YOU MUST NOT commit or push without explicit user request.
- Code fences must declare a language (`bash`, `python`, `json`, ...).
- If a quality check fails, fix the underlying issue. Do not bypass with `--no-verify`.

## Architecture map

- `src/api/`: HTTP handlers only, no business logic
- `src/services/`: all domain logic lives here
- `src/db/`: EF Core migrations and DbContext

## Workflow preferences

- Small fixes get a minimal diff. Do not rewrite a section to fix a typo.
- All new features require a unit test alongside the implementation.
- Run `npm test` before marking any task done.
```

File Locations and Precedence

Claude reads CLAUDE.md files from multiple locations and merges them in order of precedence. Understanding the hierarchy lets you share global preferences across all projects while keeping project-specific rules isolated.

Location	Scope
`~/.claude/CLAUDE.md`	Global instructions that apply to every project on your machine
`<project-root>/CLAUDE.md`	Project-level instructions, checked into the repository
`<subfolder>/CLAUDE.md`	Scoped instructions for a specific part of the codebase

The project root CLAUDE.md is the most commonly used layer. The global file is the right place for personal preferences such as preferred shell, editor, or response style, the things you want across all workspaces.

File Mentions

File mentions are the inline way to pull specific files into the active context window. Type `@` followed by a path to attach any file to your message. Claude reads it as part of the prompt rather than having to discover it through tool calls.

```
@src/services/auth.ts Why does this function re-fetch the user on every request?
```

File mentions work for source files, config files, logs, and even images. They are useful when you want to direct Claude's attention to a specific file without describing its contents, because the file speaks for itself.

Examples That Pin the Output Shape

Prose rules describe intent; examples remove ambiguity. When a project has a recurring output shape (a pagination envelope, a log line, a commit message format), two complete input and output pairs in CLAUDE.md do more than a paragraph describing it. Without them Claude will invent a plausible variant, and plausible variants differ between sessions and between teammates.

The examples belong under a conventions heading and must use the exact field names and types the real code uses, because Claude copies what it sees rather than what the surrounding sentence claims.

````
markdown
## Conventions

### Pagination shape

Every paginated endpoint returns this shape. No exceptions.

GET /todos?page=2&size=10

{ "data": [ ... ], "page": 2, "size": 10, "total": 47, "hasNext": true }

GET /users?page=1&size=5

{ "data": [ ... ], "page": 1, "size": 5, "total": 3, "hasNext": false }

`hasNext` is `page * size < total`, computed from the total count.
````

Two examples beat one, because a single example leaves the model guessing which parts are the pattern and which are the data. The second example varies the values while holding the shape fixed, which is what makes the shape readable as the rule.

Memory

Claude Code reads context from three locations before every session. The project CLAUDE.md lives in the repository root and is shared with the whole team. The global CLAUDE.md applies to every project on your machine. The user memory file is written dynamically: Claude updates it when you ask it to remember something.

Running `/memory` opens the memory file, where Claude or you can record anything that should survive a conversation reset: preferred patterns, recurring decisions, team conventions, or notes about in-progress work. Memory is injected into the context at the start of each session alongside the CLAUDE.md files. It complements

CLAUDE.md: memory is for evolving, personal notes, while CLAUDE.md is for stable, shared project rules.

You can ask Claude to remember something mid-session, and it writes it to the memory file immediately:

```
remember that I prefer direct answers without preamble and I never want trailing comments in code

remember we decided to use Zustand for global state in this project and not to introduce Redux

remember I'm currently migrating the auth service, don't touch UserController until I say so
```

Include, Exclude, Compress

Beyond CLAUDE.md and file mentions, Claude Code gives you three levers for controlling context size. You can include additional files and directories explicitly, exclude noisy paths (build outputs, lock files, generated assets) so they never appear in tool results, and compress long conversations with `/compact` when the context window fills up. These three mechanisms keep sessions fast and focused as projects grow.

Command	Usage
`/memory`	Open or edit the memory file to record facts for future sessions
`/include`	Add files to the active context window
`/exclude`	Exclude files from the context window
`/compact`	Compress conversation history when the window fills up

CHAPTER 5

Keep the Prompt Cache Warm

Environment: any Claude Code session · Where to find it: `/usage` for the token counts, `/context` for what is filling the window

The model remembers nothing between requests. Every turn you send, Claude Code re-sends the whole context: the system prompt, your project context, every prior message and tool result, and your new message. Prompt caching is how the API avoids reprocessing the part that did not change, and it is the single largest cost and latency lever in a long session.

Claude Code manages the cache for you, so there is nothing to switch on. What is worth knowing is which of your own actions throw the cache away, because each one buys you a slower, more expensive turn while the prefix rebuilds. Most of them are avoidable once you know they cost something.

How the Cache Is Organized

The API caches by matching the start of each request, called the prefix, against content it recently processed. The match is exact, so a change anywhere in the prefix recomputes everything after it. There is no per-file or per-segment caching. Claude Code therefore orders each request so the content that rarely changes comes first.

Layer	Content	Changes when
System prompt	Core instructions, tool definitions, output style	The set of loaded tool definitions changes, or Claude Code is upgraded
Project context	CLAUDE.md, auto memory, unscoped rules	Session starts, or after `/clear` or `/compact`
Conversation	Your messages, Claude's responses, tool results	Every turn

A change in the conversation layer leaves the two layers above it cached, which is the normal case: you append a message, everything before it is read from cache. A change in the system prompt invalidates all three, because every later token now sits behind a different prefix.

```
System prompt  ->  Project context  ->  Conversation  ->  Your new message

Normal turn:          all three layers read from cache
Cache key changed:    all three layers rewritten
```

Two settings are part of the cache key without appearing in the prompt text at all. Each model has its own cache, so switching models recomputes the request even when every token is identical. Each effort level has its own cache for the same model, which is why Claude Code asks you to confirm an effort change once a conversation is under way.

What a Cache Hit Is Worth

Every API response reports two token counts, and the ratio between them tells you whether caching is working.

Field	Meaning
`cache_creation_input_tokens`	Tokens written to the cache this turn, billed at the cache write rate
`cache_read_input_tokens`	Tokens served from cache this turn, billed at roughly 10% of the standard input rate

A high read-to-creation ratio is healthy. If creation stays high turn after turn, something in your prefix keeps changing, and the tables below are the list of usual suspects. Cache writes cost more than plain input tokens, which is why an avoidable invalidation is charged twice: once to re-read the history and once to write the new prefix.

Actions That Break the Cache

Each of these costs one slower, more expensive turn, after which the new prefix is cached.

Action	What happens
`/model` switch	Each model has its own cache. The next request reads the whole history with no hits
`/effort` change	Effort level is part of the cache key. Claude Code confirms first once a conversation has started
Turning on fast mode	Adds a request header that is part of the cache key. Costs once per conversation, so turning it on early is cheaper
Connecting or disconnecting an MCP server	Only when its tools load into the prefix. Deferred tools append and leave the cache intact
Enabling or disabling a plugin	Only when the plugin provides MCP servers. Skills, commands, agents, hooks and themes are cache-safe
Denying an entire tool	A bare deny rule such as `Bash` removes the tool from the system prompt. Scoped rules like `Bash(rm *)` do not
`/compact`	Replaces the history with a summary, so the conversation layer no longer shares a prefix with the old one
Upgrading Claude Code	A new version usually changes the system prompt or tool definitions. Resuming a long session after an upgrade is the most expensive request you can send

Plan mode is cache-safe on its own, because its instructions are appended as conversation messages. The exception is the `opusplan` setting, which resolves to Opus in plan mode and Sonnet during execution, so every plan-mode toggle is a model switch.

Actions That Keep the Cache

Action	Why it is safe
Editing files in your repository	Reads append to the conversation. Claude Code appends a note that the file changed rather than rewriting the earlier read
Editing CLAUDE.md mid-session	Root and user CLAUDE.md are read once at session start and held in memory. The cache survives, but the edit does not apply either
Changing output style	Part of the system prompt, read at session start, so the change waits for the next `/clear` or restart
Changing permission mode	Mode changes touch neither the system prompt nor the tool definitions

Action	Why it is safe
Invoking a skill or command	Both inject their instructions as user messages at the point of invocation
`/recap`	Appends the summary as command output instead of replacing the history, unlike `/compact`
`/rewind`	Truncates back to a prefix the cache was already built from, so the next request hits an earlier entry
Spawning a subagent	The subagent builds its own cache. From the parent's side the call and result are appended

The CLAUDE.md row is the one that surprises people. A mid-session edit is silently inert: Claude keeps working from the version loaded at session start, and the new content loads on the next `/clear`, `/compact` or restart. Nested CLAUDE.md files in subdirectories behave differently, because they load later, when Claude first reads a matching file.

Lifetime, Scope, and Forks

Cached prefixes expire after a period of inactivity, and every request that hits the cache resets the timer. The cache stays warm as long as you keep working, which is why the first turn back after a long break is noticeably slower. The API offers a five-minute TTL and a one-hour TTL that bills cache writes at a higher rate. Claude Code picks the TTL from how you authenticate: a Claude subscription requests the one-hour TTL automatically, while an API key defaults to the cheaper five-minute TTL.

In Claude Code the cache is effectively scoped to one machine and one directory. The system prompt embeds the working directory, platform, shell, OS version and auto-memory paths, so two sessions in different directories build different prefixes and miss each other's cache. That includes worktrees of the same repository, which is a cost worth knowing before Chapter 8.

A subagent starts its own conversation with its own system prompt and tool set, so it builds its own cache from zero. A fork is the opposite: it inherits the parent's system prompt, tools and conversation history exactly, so its first request reads the parent's cache instead of building a new one.

Check Cache Performance

`/usage` reports the two token counts for the current session, broken out per model:

```
Usage by model:
   claude-sonnet-4-6:  1.2k input, 5.3k output, 940.0k cache read, 50.0k cache write ($0.55)
```

On a paid plan, `/usage` also flags behaviours that account for 10% or more of recent usage, cache misses among them. For a live reading rather than a session total, a statusline script can read the two counts straight off the `current_usage` object.

Command	Usage
`/usage`	Read cache read against cache write for the session; `/cost` is an alias
`/context`	See which layer is filling the window before deciding whether to compact
`/rewind`	Abandon a bad path by truncating to an already-cached prefix instead of compacting
`/effort`	Set effort once at the top of a session, since a mid-session change recomputes everything

CHAPTER 6

Select the Right Model

Environment: any Claude Code session · Where to find it: `/model` to switch, `/fast` to toggle fast mode, `/usage` for the bill

Available Models

Model	Best for
Opus 5 (default)	Complex reasoning, multi-step planning, challenging architectural decisions, agentic coding
Opus 5 (1M context)	The same, with the extended context window selected explicitly, billed as extra usage
Sonnet 5	Everyday tasks, general-purpose coding, standard workflows, best value
Haiku 4.5	Fastest responses, quick answers, simple lookups, tight token budgets (200K context)

Opus 4.8, Opus 4.7, and Sonnet 4.6 remain selectable for teams that pinned an older model, but new work should start on the Claude 5 family.

Fast Mode

Fast mode runs Claude Opus with faster output rather than downgrading to a smaller model. Toggle it with `/fast`. It is available on Opus 5, Opus 4.8, and Opus 4.7, and it is priced above standard Opus usage, so reach for it when turnaround matters more than token spend.

Token Usage and Cost Awareness

Defaulting to the strongest model on every task wastes tokens and increases costs unnecessarily. Haiku handles most everyday coding tasks efficiently and costs a fraction of Opus, while Sonnet provides an excellent middle ground for moderately complex work. Save Opus for situations that genuinely require its advanced reasoning, such as complex system design or intricate debugging. Matching model strength to task complexity optimizes both spending and response latency without sacrificing quality.

Working Past Your Plan Limit

Subscription plans meter usage against a per-seat allowance rather than a token balance. When you hit it, usage credits let you keep working, and `/usage-credits` opens the right place for your role: billing settings for a Pro or Max subscriber, the organization's usage settings for a Team or Enterprise member with billing access, and a request to your admins for everyone else. The command needs a claude.ai login through `/login` and is unavailable with API key authentication.

Run `/usage` to see where you stand. On a paid plan it also breaks recent usage down by skill, subagent, plugin, and MCP server, and flags any behaviour accounting for 10% or more of it.

The Other Half of the Bill

Tier selection is one lever, and it is the smaller one in a long session. What you send every turn matters more than which model reads it, because the whole conversation ships with every message. Chapter 5 covers the mechanism that makes that affordable and the actions that throw it away, including the `/model` switch this chapter is about.

Command	Usage
`/model`	Switch the active model mid-session without restarting
`/fast`	Toggle fast mode on an Opus model for higher output speed
`/usage`	Show token usage and estimated cost for the current session
`/status`	Display current model, context usage, and session info

CHAPTER 7

Configure the Permission Model

Environment: terminal, VS Code · Where to find it: `.claude/settings.json` in the repository, `/permissions` to inspect the active rules

Claude Code's permission system controls what Claude can access and execute in your development environment. Unlike a chat interface that only reads and writes text, Claude Code can invoke tools to read files, run shell commands, and interact with your system.

The single most important fact comes first: permission rules are enforced by Claude Code, not by the model. Instructions in your prompt or in `CLAUDE.md` shape what Claude tries to do, but they never change what Claude Code allows. `CLAUDE.md` is guidance; `.claude/settings.json` is enforcement, and only one of the two survives a model that misreads you.

The Three Permission Tiers

Claude Code sorts every tool call into one of three tiers, and each tier has a different default answer to the question of whether it needs your approval.

Tier	Example	Approval required	What "don't ask again" does
Read-only	File reads, Grep, Glob	No, inside the working directory	Not applicable
Bash commands	Shell execution	Yes, except a built-in read-only set	Saves permanently, per repository and command
File modification	Edit and Write	Yes	Lasts until the session ends

The tiers explain the friction you feel in a fresh project. Reading is free, so Claude explores without interrupting you. Editing always asks, and the approval expires with the session. Bash sits in between: a built-in set of read-only commands runs silently, and everything else prompts until you write a rule.

Claude Code recognises a fixed set of shell commands as read-only and runs them without a prompt in every mode: `ls`, `cat`, `echo`, `pwd`, `head`, `tail`, `grep`, `find`, `wc`, `which`, `diff`, `stat`, `du`, `cd`, and the read-only forms of `git`. The set is not configurable. This is why `git status` runs silently in a fresh project while `dotnet build` does not, and it is worth knowing before you conclude that a rule is broken.

Writing Permission Rules

Rules live in a `permissions` object in a settings file and take the form `Tool` or `Tool(specifier)`. There are three lists:

```json
{
  "permissions": {
    "allow": [
      "Bash(dotnet build)",
      "Bash(git diff *)"
    ],
    "ask": [
      "Bash(git push *)"
    ],
    "deny": [
      "Read(./.env)",
      "Bash(rm -rf *)"
    ]
  }
}
```

Rules are evaluated deny, then ask, then allow, and the first match in that order wins. Specificity never changes the order, which has one consequence worth internalising: a deny rule cannot carry exceptions. Denying `Bash(git push *)` while allowing `Bash(git push origin main)` blocks every push, including the one you meant to permit.

A bare tool name behaves differently from a scoped one. Denying `Bash` removes the tool from Claude's context entirely, so Claude never sees it and never proposes a command. Denying `Bash(rm *)` leaves the tool available and blocks matching calls as they are attempted.

Bash patterns

Bash rules accept `*` at any position, and the space before it matters. `Bash(ls *)` enforces a word boundary and matches `ls -la` but not `lsof`, while `Bash(ls*)` matches both. The `:*` suffix is an equivalent way to write a trailing wildcard, recognised only at the end of a pattern.

Two behaviours catch people out. Compound commands are checked per subcommand, so `Bash(dotnet build)` does not permit `dotnet build && dotnet run`. And a wildcard spans arguments freely, so `Bash(dotnet *)` also allows `dotnet nuget push`. Grant the narrowest pattern that covers your real usage.

Claude Code strips a fixed set of wrappers before matching, so `Bash(dotnet test *)` also covers `timeout 30 dotnet test`. The stripped wrappers are `timeout`, `time`, `nice`, `nohup`, `stdbuf`, the builtins `command` and `builtin`, and zsh's `noglob`. Environment runners such as `npx` and `docker exec` are deliberately not stripped, because a rule like `Bash(docker exec *)` would otherwise permit whatever command follows it.

File path patterns

`Read` and `Edit` rules use gitignore pattern syntax with four anchors, and confusing them is the most common way to write a rule that protects nothing:

Pattern	Anchored at	Example
`//path`	Filesystem root	`Read(//Users/alice/secrets/**)`
`~/path`	Home directory	`Read(~/.aws/**)`
`/path`	The settings file's own location	`Read(/appsettings.Secrets.json)`
`path` or `./path`	Current directory	`Read(./*.env)`

A single leading slash is not an absolute path: `/Users/alice/file` anchors at the project root. `Edit` rules cover every file-editing tool, and a `Read` deny also blocks editing the same path.

Path rules written for `Write`, `NotebookEdit`, or `Glob` are accepted by the parser but never matched by the file permission checks, so they look like protection and provide none. Claude Code warns at startup for each one, naming the replacement to use. Read the first lines of a session's output after changing a settings file.

Read and Edit deny rules also cover the file commands Claude Code recognises inside Bash, including `cat`, `head`, `tail`, and `sed`, so denying a path closes the obvious shell bypass at the same time. They do not cover arbitrary subprocesses: a Python or Node script that opens the file itself still reads it. For enforcement across every process, enable the sandbox.

Permission Modes

A mode sets the default answer before any rule is consulted. Set one with `defaultMode` in a settings file, or switch mid-session.

Mode	Behaviour
`default`	Prompts on first use of each tool. Shown as Manual in the CLI
`acceptEdits`	Auto-accepts file edits and common filesystem commands in the working directory
`plan`	Reads and explores but does not edit your source files
`auto`	Auto-approves with background safety checks against your stated request
`dontAsk`	Auto-denies anything not pre-approved by an allow rule
`bypassPermissions`	Skips prompts entirely. Only for containers and throwaway VMs

`bypassPermissions` still honours explicit `ask` rules, and still prompts as a circuit breaker for removals targeting the filesystem root or your home directory. Organizations can block it outright with `permissions.disableBypassPermissionsMode`.

Where Permissions Are Stored

Claude Code reads permission rules from several files and merges them by precedence, from highest to lowest:

Precedence	Source	Committed to git
1	Managed settings, deployed by an organization	Not applicable
2	Command line arguments, for the session only	No
3	`.claude/settings.local.json`	No
4	`.claude/settings.json`	Yes
5	`~/.claude/settings.json`	No

That ordering decides which allow rule wins. It does not let a lower file weaken a deny: if any level denies a tool, no other level can allow it, because deny rules from every scope are evaluated before allow rules. A teammate cannot quietly re-enable `rm -rf` for themselves in a local file.

Use `.claude/settings.json` for policy the whole team should share and commit it. Use `.claude/settings.local.json` for personal convenience that stays off version control. This is also where Claude Code saves your own choices: selecting "Yes, don't ask again" on a Bash prompt writes the rule to the local file at the repository root.

Workspace Trust

Project allow rules only take effect in a workspace you have trusted. Run Claude Code interactively in a directory once and accept the trust dialog. Until you do, every allow entry from that project is ignored and a startup line tells you so. Deny rules are unaffected, which is the safe way round, but it means a policy can look inert for a reason that has nothing to do with your patterns.

Denials Are a Feature

When Claude requests something you want to refuse, refuse it. Claude finds another approach or explains why the operation is needed, and you stay in control. The judgement to build is which decisions belong in a prompt at all: an approval you click twenty times an hour is not a control, it is a reflex, and the prompt that finally matters gets the same reflex.

That is the real argument for writing rules. Pre-approve the narrow, non-destructive commands you run all day so the remaining prompts are few enough to read. Deny the handful whose worst case is unrecoverable so there is no prompt to approve by accident. Everything else keeps asking.

Command	Usage
`/permissions`	List every active rule and the settings file each one came from, and edit them
`/update-config`	Edit `settings.json` through guided prompts rather than by hand
`/status`	Show the active session's model, login, and working directories
`/doctor`	Health check the installation, including settings files that fail to parse

CHAPTER 8

Run Parallel Agents with Git Worktrees

Environment: terminal, VS Code · Where to find it: `git worktree` in a shell, the `--worktree` flag on `claude`, and `.claude/worktrees/`

A git worktree lets you check out a separate branch of your repository into a completely independent directory while sharing the same underlying `.git` store. Unlike switching branches in your current checkout, each worktree has its own working tree: its own files, its own staged changes, and its own untracked file state. You can have a feature branch open in one directory and a hotfix branch open in another at the same time, without stashing or losing work.

```bash
# Add a worktree for a feature branch
git worktree add ../my-feature feature/my-feature

# List all active worktrees
git worktree list

# Remove a worktree when done
git worktree remove ../my-feature
```

Manual, Flag, and Automatic

There are three ways to work with worktrees in Claude Code. The manual approach uses `git worktree add` directly in a terminal: you create the worktree, run `claude` inside it, and remove the worktree yourself when finished. Use it when you need a specific existing branch or a path outside the repository.

The flag approach is the everyday one. Pass `--worktree <name>` (or `-w`) and Claude Code creates the worktree at `.claude/worktrees/<name>/` on a branch named `worktree-<name>`, then starts the session there:

```bash
claude --worktree feature-auth
```

The automatic approach needs no flag at all. Background sessions started with `--bg` move themselves into an isolated worktree before touching any file, and a subagent does the same when its frontmatter carries `isolation: worktree`. In all three cases Claude can also enter or leave a worktree mid-session through the `EnterWorktree` and `ExitWorktree` tools. A plain interactive `claude` with no flag stays in your current checkout and creates nothing.

How Claude Code Uses Worktrees

Before making any code changes in an agentic task, Claude calls the `EnterWorktree` tool to check out a fresh branch into an isolated directory. This protects your working copy from interference while you continue editing in your own checkout.

When Claude finishes, the worktree is automatically removed if no files were changed. If changes were made, the worktree path and branch name are returned so you can review the diff or merge the work. This pattern makes it safe to run multiple Claude agents in parallel: each operates in its own worktree and cannot clobber files another agent or you are editing.

```
EnterWorktree  ->  edit files on isolated branch  ->  ExitWorktree  ->  report branch name
```

A typical scenario: you are on a feature branch and ask Claude to fix a production bug. Claude enters a worktree on `hotfix/bug-123`, edits and commits there, exits, and reports the branch. You review the diff and merge on your own schedule, while your feature work continued uninterrupted and never conflicted.

Cleaning Up

Removal refuses in two situations, and each prints a different error. A worktree Claude Code is actively using is locked, so `git worktree remove` reports the lock and names the session holding it; release it with `git worktree unlock` first. A worktree holding modified or untracked files refuses too, and only `--force` overrides that, because git will not silently discard work.

Removing a worktree leaves its branch behind, so a full cleanup is two commands. A periodic sweep also removes worktrees Claude created for subagents and background sessions once they pass your `cleanupPeriodDays` setting, skipping any that still hold changes or unpushed commits. It never touches worktrees you created with `--worktree`.

Carrying Local Files Into a Worktree

A worktree is a fresh checkout, so gitignored files such as `.env` are absent and your app fails to start in it. List them in a `.worktreeinclude` at the repository root, using `.gitignore` syntax, and Claude Code copies them into every worktree it creates:

```
.env
.env.local
config/secrets.json
```

Only files that match a pattern and are also gitignored are copied, so tracked files are never duplicated. Add `.claude/worktrees/` to your `.gitignore` as well, or every worktree shows up as untracked noise in your main checkout.

Command	Usage
`/review`	Review changes made in a worktree branch before merging
`/rewind`	Restore code, conversation, or both to an earlier prompt in the session
`/status`	Confirm which directory and branch the session is working in

CHAPTER 9

Supervise Background Sessions with the Agents View

Environment: terminal · Where to find it: `claude agents`, or press the left arrow key on an empty prompt inside any running session

A single Claude session occupies your terminal and your attention. The agents view breaks that one-to-one link: sessions run in the background, and you move between them from one screen, checking on work rather than waiting for it. The shift is from operating a tool to supervising a small team, and the skill it demands is triage, not typing. It requires Claude Code v2.1.139 or later.

Dispatch Work to the Background

A background session starts, runs, and finishes without holding your terminal. Start one from the shell with `--bg`, or send the conversation you are already in to the background with `/background`.

```bash
claude --bg "fix the flaky SettingsChangeDetector test"
claude --bg --name "flaky-test-fix" "investigate the SettingsChangeDetector timeout"
claude --agent code-reviewer --bg "address the review comments on this branch"
```

The command returns immediately, printing a short session ID and the four commands that manage it:

```
backgrounded - 7c5dcf5d
  claude agents             list sessions
  claude attach 7c5dcf5d    open in this terminal
  claude logs 7c5dcf5d      show recent output
  claude stop 7c5dcf5d      stop this session
```

Naming a session with `--name` is worth the extra words once you have more than three running, because the default label is the prompt itself and long prompts all look alike in a list.

Read the Board

Sessions are grouped by state, and the ordering is the point: the group that needs you sits at the top.

Group	Meaning
Pinned	Sessions you marked with `Ctrl+T` so they stay running while idle
Ready for review	Finished work waiting for you to look at it
Needs input	Blocked on a permission prompt or a question
Working	Actively running tools or generating
Completed	Finished and no longer holding anything

A second mark shows whether the process itself is alive. An animated star means the session replies immediately, a dim dot means the process exited but you can still peek, reply, or attach, and a separate mark flags a `/loop` session sleeping between iterations. When the list grows past a screen, filter it by typing in the

dispatch input rather than scrolling: `s:blocked` for everything waiting on you, `a:code-reviewer` for sessions running that named agent, `#1234` for the session working on that pull request.

Work Without Leaving the Board

The view is a control surface, not a list. Most triage never requires attaching to a session at all.

Key	Action
`Space`	Peek at the session's latest output in a panel
`Enter`	Attach, or dispatch a new session if you typed a prompt
Right arrow	Attach to the selected session
`Shift+Enter`	Dispatch and attach immediately
`Ctrl+T`	Pin or unpin, keeping the process running while idle
`Ctrl+R`	Rename the session
`Ctrl+S`	Switch grouping between state and directory
`Ctrl+X`	Stop the session; press again within two seconds to delete
Left arrow	Detach and return to the board

Peeking is the habit worth building. From the peek panel you can type a reply and send it, press `Tab` to accept a suggested answer, or answer a multiple-choice question by pressing its number, all without losing your place in the list. Detaching never stops a session.

How File Edits Stay Isolated

Parallel sessions would be useless if they overwrote each other, so before a background session edits any file, Claude moves it into an isolated git worktree under `.claude/worktrees/`. Every session reads the same repository history and writes to its own branch. This is the mechanism from Chapter 8, applied automatically rather than on request.

Claude skips the worktree in three cases: the session is already inside a linked worktree, the working directory is not a git repository, or the write lands outside the working directory. You can turn the behaviour off, though doing so puts parallel sessions back in each other's way.

```json
{
  "worktree": {
    "bgIsolation": "none"
  }
}
```

Deletion is where work gets lost. Pressing `Ctrl+X` twice in the agents view removes the session's worktree along with any uncommitted changes in it, while `claude rm` from the shell keeps a worktree that still holds uncommitted changes. Both keep a worktree with unpushed commits, and neither touches worktrees you created yourself. Commit or push anything you want before deleting.

The Supervisor Behind It

Background sessions outlive the terminal that started them because a separate per-user supervisor process owns them. It keeps one pre-warmed worker ready so dispatch feels instant, restarts a session whose process dies unexpectedly, and reconnects after your machine wakes from sleep. Idle sessions that are not pinned stop after roughly an hour to free resources.

Sessions are local to your machine, so they survive sleep but not a shutdown. Supervisor state lives under `~/.claude/`, which is where to look when something is stuck:

Path	Holds
`~/.claude/daemon.log`	Supervisor log
`~/.claude/daemon/roster.json`	The list of running sessions
`~/.claude/jobs/<id>/state.json`	Per-session state

Two commands cover the rest: `claude daemon status` reports the supervisor's state and version, and `claude daemon stop --any` stops it along with every background session.

Know the Cost Before You Fan Out

Ten parallel agents consume roughly ten times the quota of one. The view makes dispatching cheap enough that this is easy to forget, and the bill arrives as rate limiting mid-afternoon rather than as an error at dispatch. Fan out when the work is genuinely parallel and each session has a clear finish line, not as a default.

Command	Usage
`/background`	Send the current conversation to the background and return to the board
`/bg`	Background the session with a follow-up instruction to carry on with
`/fork`	Copy the current conversation into a second session, optionally with a new task
`/model`	Set the model used for sessions dispatched from the agents view

CHAPTER 10

Turn Session Output into a Shareable Page

Environment: terminal, Claude Desktop · Where to find it: `Ctrl+]` reopens the most recent artifact, the gallery lives at claude.ai/code/artifacts

Most of what Claude produces belongs in the terminal. Some of it does not: four layout options you want to see next to each other, a diff with reasoning written beside the lines, a migration checklist that fills in while a long task runs. An artifact is a live web page published from your session to a private URL on claude.ai, and it stays at that URL as the session keeps revising it.

The feature is on by default for a session signed in with `/login`, so you will meet it before you go looking for it. It requires Claude Code v2.1.183 or later, or the desktop app v1.13576.0 or later.

What an Artifact Is, and What It Is Not

An artifact is a capture of work, not an application. It is one self-contained page with no backend, so it cannot store what someone types into it, cannot serve a second route, and cannot call an API of its own. Its only path to outside data at view time is calling MCP connectors, which Module 4 covers.

That boundary is the useful part rather than a limitation to work around. A page that is a snapshot can be published, versioned, shared, and thrown away without operating anything. For a hosted internal tool with a backend, deploy on your own infrastructure instead.

Publish One

Ask for a page in plain language, or describe visual output and let Claude decide. Claude writes the page to an `.html` or `.md` file in your project, then publishes that file.

```
Build a dashboard artifact of last week's deploy failures by service
and keep it updated as you investigate.
```

The first publish of a new artifact asks for permission, naming both the title and the file it is about to publish. Approve it and Claude prints the URL, your browser opens the page, and republishing that same artifact never asks again. Claude picks the title and a browser-tab emoji, both of which show up in your gallery and in any link you share, so name them in the prompt when you care.

```
Ask for a page  ->  Claude writes page.html  ->  permission prompt (first publish only)
   ->  URL printed, browser opens  ->  ask for a revision
   ->  republished to the same URL as a new version
```

Update It, and Choose What Viewers See

Ask for a revision and Claude edits the underlying file and publishes again to the same URL. Anyone with the page open sees the change in place, which is what makes an artifact worth keeping open during a long task.

Every publish becomes a version. The Share control in the page header holds a version picker and an option to always share the latest, so a page can keep moving while reviewers stay pinned to the version you sent them.

A different session does not know about your artifact. Give it the URL and it revises the existing page; leave the URL out and you get a second artifact instead of a new version of the first.

Share It

A new artifact is visible only to you, and sharing is deliberate. The page header names you as the author and links to your gallery, so whoever opens it knows where it came from.

Plan	Audience you can grant
Pro and Max	A public link that anyone on the internet can open, which is the only sharing path on these plans
Team and Enterprise	Named people in your organization or everyone in it, with public links off until an Owner enables them

On Team and Enterprise plans a person you share with can be promoted from viewer to editor. An editor republishes the same way you update from another session: they paste the artifact URL into their own session, and Claude pulls the current content and publishes their changes as a new version.

What Is Worth Building

The patterns below come up far more often than anything else, and each one is a prompt rather than a feature you enable.

Pattern	Ask for
Walk through a change	A rendered diff with margin annotations and findings color-coded by severity
Compare alternatives	Four distinctly different variants in a grid with a one-line tradeoff under each
Tune with controls	Sliders and toggles bound to the values you are adjusting, with the result rendered live
Bring the result back	A control that emits text you paste into the terminal, so the decision returns to the session
Track work in progress	A checklist the session ticks off and annotates while a long task runs

The fourth one is the one people miss. A triage board with a "Copy as prompt" button turns the page from a report into an input surface, because the ordering you dragged into place becomes the next prompt.

Make the Page Look Like Your Product

Claude applies a built-in design skill when it builds an artifact, so pages arrive with a deliberate palette, typography, and layout without being asked. That skill looks for an existing design system in your project first. Record your tokens where Claude already reads, such as `CLAUDE.md` or a theme file in the repository:

```markdown
## Design system

- Colors: primary #1a4d8f, accent #f59e0b, surface #f8fafc
- Typography: Inter for body, JetBrains Mono for code
- Spacing: 8px scale, 6px border radius
```

Precedence runs prompt, then your design system, then Claude's own choices. The heading above is an example rather than a required schema: any clear list of colors, fonts, and spacing works.

Page Constraints

Claude Code wraps your file in an HTML shell and serves it under a strict Content Security Policy. The constraints below are what that policy and the single-page model produce.

Constraint	Effect
No external requests	Scripts, stylesheets, fonts, and images from any other host are blocked, along with `fetch`, XHR, and WebSockets. Claude inlines CSS and JavaScript and embeds images as data URIs
No backend	The page cannot store form input or authenticate viewers itself
Single page	Relative links do not resolve, so multi-section content uses in-page anchors
Source file types	The published file must be `.html`, `.htm`, or `.md`, and Markdown renders as styled HTML
Rendered size	16 MiB or smaller, and a large embedded raster image is the usual cause of a size failure

A styled page costs more output tokens than the same content as terminal text, with data-URI images the dominant cost. Prefer SVG or plain HTML and CSS for diagrams, skip interactivity you do not need, and have the page summarize a large dataset rather than inline it.

Availability and Turning It Off

Artifacts need every condition below at once. When one is missing, Claude writes a local HTML file or says it cannot publish, which is the symptom to recognise rather than debug blindly.

Requirement	Available when
Plan	Pro, Max, Team, or Enterprise, with Enterprise requiring an Owner to enable it
Authentication	The session is signed in with `/login`, never an API key, gateway token, or cloud-provider credential
Model provider	The Anthropic API, not Amazon Bedrock, Google Vertex AI, or Microsoft Foundry
Organization policy	CMEK, HIPAA, and Zero Data Retention are not enabled
Surface	Claude Code v2.1.183 or the desktop app v1.13576.0, and off by default in the Agent SDK, GitHub Action, and MCP-server contexts

To switch artifacts off for your own sessions regardless of the organization setting, use whichever layer you already manage:

```json
{
  "disableArtifact": true,
  "permissions": {
    "deny": ["Artifact"]
  }
}
```

Command	Usage
`/login`	Sign the session in to a claude.ai account, the requirement an API-key session cannot meet
`/memory`	Open the `CLAUDE.md` that holds your design tokens
`/permissions`	Add `Artifact` to the deny list when a repository should never publish anything outward

Command	Usage
`/usage`	Check what a styled, image-heavy page actually cost in output tokens

CHAPTER 11

Labs: Install, Build, and Prompt

Environment: terminal, VS Code · Where to find it: the `labs/01-fundamentals/` folder of the course repository

Three labs close this module. Lab 01 is a prerequisite rather than a teaching lab: nothing else in the course works until Claude Code is installed and authenticated. Lab 02 is the core lab, the one to run if you run nothing else. Lab 03 goes deeper into prompting and ends with a real marketplace plugin.

Lab 01: Install and Set Up Claude Code

Required, beginner, about 30 minutes. Install the Claude Code CLI, authenticate it, and explore it through the VS Code extension. You finish with a verified installation, a working login, and both extensions in place, which is the baseline every later module assumes.

Lab 02: Build a Task Tracking REST API

Core, beginner, about 40 minutes. Build a Python REST API for task management using Claude Code. Every step is prompt-driven: you provide the prompt, Claude does the work. The lab is the first real taste of delegation, where the quality of your briefing decides the quality of the result.

Lab 03: Apply Prompting Techniques and Install a Plugin

Optional, intermediate, about 50 minutes. Apply five prompting techniques to your harness: statefile maintenance, anchors, `@`-mentions, few-shot examples, and slash-command promotion. Finish by installing a real plugin from the Claude Code marketplace and observing how it modifies your harness.

The lab has a starter you scaffold yourself and a finished counterpart beside it. The starter stays pristine, with no dependencies and no build output. Install, run, and generate only inside the solution folder, and use it as the completed reference when you want to compare.

Time Budget

Path	Labs	Time
Core path	Lab 01 and Lab 02	1h 10m
All labs	Lab 01, Lab 02, and Lab 03	2h

Where This Module Leads

Everything here is a foundation for the modules that follow. The harness you started in Chapter 4 grows into skills, subagents, and hooks. The permission rules from Chapter 7 become the safety boundary for headless and CI runs. The worktrees and background sessions from Chapters 8 and 9 are the mechanism behind every parallel workflow later in the course, and the artifacts from Chapter 10 return in Module 4 as pages that pull live MCP data at view time.